

Programação dinâmica

Aula 9

Fábio Henrique Viduani Martinez

Faculdade de Computação
Universidade Federal de Mato Grosso do Sul

Projeto e Análise de Algoritmos I

Conteúdo da aula

- 1 Introdução
- 2 Número de Fibonacci
- 3 Corte de barras de aço
- 4 Multiplicação de uma sequência de matrizes
- 5 Maior subsequência comum
- 6 Elementos de programação dinâmica
- 7 Exercícios

Motivação

- ▶ programação dinâmica e o método da divisão-e-conquista solucionam problemas através da combinação de soluções de subproblemas
- ▶ programação dinâmica, diferentemente da divisão-e-conquista, é aplicada quando os subproblemas não são independentes e compartilham subsubproblemas
- ▶ um algoritmo de programação dinâmica soluciona cada subsubproblema exatamente uma vez e então salva a resposta em uma tabela, evitando o trabalho de recomputar a resposta toda vez que o subsubproblema é encontrado
- ▶ “programação” nesse contexto se refere ao método tabular

Motivação

- ▶ programação dinâmica é aplicada a **problemas de otimização**: cada possível solução tem um valor e queremos encontrar uma solução com o valor ótimo (mínimo ou máximo)
- ▶ chamamos tal solução **uma** solução ótima do problema
- ▶ o desenvolvimento de um algoritmo de programação dinâmica segue quatro passos:
 - ▶ caracterizar a estrutura de uma solução ótima
 - ▶ definir recursivamente o valor de uma solução ótima
 - ▶ computar o valor de uma solução ótima
 - ▶ construir uma solução ótima a partir da informação computada

Definição

- ▶ a **sequência de Fibonacci** é descrita pela seguinte fórmula:

$$F_n = \begin{cases} n, & \text{se } n \leq 1 \\ F_{n-1} + F_{n-2}, & \text{se } n \geq 2 \end{cases}$$

- ▶ a tabela mostra os valores de F_n para $n = 0, \dots, 9$:

n	0	1	2	3	4	5	6	7	8	9
F_n	0	1	1	2	3	5	8	13	21	34

Definição

Problema da determinação do número de Fibonacci

Dado um número inteiro $n \geq 0$, computar o n -ésimo número de Fibonacci F_n

Algoritmo recursivo

RECURSIVE-FIBONACCI(n)

01. **se** $n \leq 1$
02. **devolva** n
03. **senão**
04. **devolva** RECURSIVE-FIBONACCI($n - 1$) + RECURSIVE-FIBONACCI($n - 2$)

Algoritmo recursivo

RECURSIVE-FIBONACCI(n)

01. **se** $n \leq 1$

02. **devolva** n

03. **senão**

04. **devolva** RECURSIVE-FIBONACCI($n - 1$) + RECURSIVE-FIBONACCI($n - 2$)

n	16	32	40	41	42	43	44	45	47
tempo (s)	0.002	0.06	2.91	4.71	7.62	12.37	19.94	32.37	84.50

Algoritmo recursivo

RECURSIVE-FIBONACCI(n)

```

01.  se  $n \leq 1$ 
02.    devolva  $n$ 
03.  senão
04.    devolva RECURSIVE-FIBONACCI( $n - 1$ ) + RECURSIVE-FIBONACCI( $n - 2$ )

```

Recorrência do tempo de execução:

$$T(n) = \begin{cases} O(1), & \text{se } n \leq 1 \\ T(n-1) + T(n-2) + O(1), & \text{se } n \geq 2 \end{cases}$$

Podemos mostrar que $T(n) = \Omega(2^n)$

“De baixo pra cima”

BOTTOM-UP-FIBONACCI(n)

01. $F[0] \leftarrow 0$

02. $F[1] \leftarrow 1$

03. **para** $i \leftarrow 2$ **até** n

04. $F[i] \leftarrow F[i-1] + F[i-2]$

05. **devolva** $F[n]$

“De baixo pra cima”

BOTTOM-UP-FIBONACCI(n)

01. $F[0] \leftarrow 0$

02. $F[1] \leftarrow 1$

03. **para** $i \leftarrow 2$ **até** n

04. $F[i] \leftarrow F[i-1] + F[i-2]$

05. **devolva** $F[n]$

Tempo de execução: $\Theta(n)$

De cima para baixo com 'memorização'

MEMOIZED-FIBONACCI(n)

01. seja $F[0..n]$ um novo vetor
02. **para** $i \leftarrow 0$ **até** n
03. $F[i] \leftarrow -1$
04. **devolva** LOOKUP-FIBONACCI(F, n)

LOOKUP-FIBONACCI(F, n)

01. **se** $F[n] \geq 0$
02. **devolva** $F[n]$
03. **senão se** $n \leq 1$
04. $F[n] \leftarrow n$
05. **senão**
06. $F[n] \leftarrow \text{LOOKUP-FIBONACCI}(F, n-1) + \text{LOOKUP-FIBONACCI}(F, n-2)$
07. **devolva** $F[n]$

De cima para baixo com 'memorização'

MEMOIZED-FIBONACCI(n)

01. seja $F[0..n]$ um novo vetor
02. **para** $i \leftarrow 0$ **até** n
03. $F[i] \leftarrow -1$
04. **devolva** LOOKUP-FIBONACCI(F, n)

LOOKUP-FIBONACCI(F, n)

01. **se** $F[n] \geq 0$
02. **devolva** $F[n]$
03. **senão se** $n \leq 1$
04. $F[n] \leftarrow n$
05. **senão**
06. $F[n] \leftarrow \text{LOOKUP-FIBONACCI}(F, n-1) + \text{LOOKUP-FIBONACCI}(F, n-2)$
07. **devolva** $F[n]$

Tempo de execução: $\Theta(n)$

Descrição

- ▶ uma empresa compra longas barras de aço e as corta em barras menores para revenda
- ▶ os cortes são realizados de tal modo que as barras resultantes têm sempre comprimento inteiro
- ▶ um corte não tem valor e não é cobrado pela empresa
- ▶ para cada $i = 1, 2, \dots$, a empresa cobra o preço p_i por uma barra de comprimento i

Exemplo

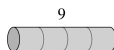
comprimento i	1	2	3	4	5	6	7	8	9	10
preço p_i	1	5	8	9	10	17	17	20	24	30

Problema

Problema do corte da barra de aço

Dadas uma barra de n metros de comprimento e uma tabela de preços p_i para $i = 1, 2, \dots, n$, determine o valor máximo de venda r_n que é possível obter pelo corte da barra e venda de seus pedaços

Exemplo para $n = 4$



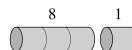
(a)



(b)



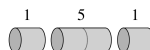
(c)



(d)



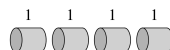
(e)



(f)



(g)



(h)

$$p_2 + p_2 = 5 + 5 = 10$$

Propriedades

- ▶ podemos cortar uma barra de comprimento n de 2^{n-1} formas diferentes
- ▶ denotamos um corte em pedaços usando a notação aditiva padrão tal que $7 = 2 + 2 + 3$ indica que uma barra de comprimento 7 foi cortada em 3 pedaços, dois de comprimento 2 e um de comprimento 3
- ▶ se uma solução ótima corta a barra em k pedaços, para algum $1 \leq k \leq n$, então uma decomposição ótima

$$n = i_1 + i_2 + \dots + i_k$$

da barra em k pedaços de comprimentos $i_1, i_2, \dots, i_k$ fornece um valor máximo de venda correspondente de

$$r_n = p_{i_1} + p_{i_2} + \dots + p_{i_k}$$

De volta ao exemplo

comprimento i	1	2	3	4	5	6	7	8	9	10
preço p_i	1	5	8	9	10	17	17	20	24	30

De volta ao exemplo

- no exemplo, podemos determinar os valores ótimos dos valores r_i por inspeção, para $i = 1, 2, \dots, n$, com as decomposições ótimas correspondentes, conforme a seguir

r_1	=	1	da solução 1 = 1 (sem cortes)
r_2	=	5	da solução 2 = 2 (sem cortes)
r_3	=	8	da solução 3 = 3 (sem cortes)
r_4	=	10	da solução 4 = 2 + 2
r_5	=	13	da solução 5 = 2 + 3
r_6	=	17	da solução 6 = 6 (sem cortes)
r_7	=	18	da solução 7 = 1 + 6 ou 7 = 2 + 2 + 3
r_8	=	22	da solução 8 = 2 + 6
r_9	=	25	da solução 9 = 3 + 6
r_{10}	=	30	da solução 10 = 10 (sem cortes)

Geral

- ▶ mais genericamente, podemos obter os valores r_n para $n \geq 1$ em termos dos valores de venda ótimos para barras mais curtas:

$$r_n = \max(p_n, r_1 + r_{n-1}, r_2 + r_{n-2}, \dots, r_{n-1} + r_1) \quad (1)$$

- ▶ p_n corresponde à barra sem corte algum, e a sua venda sem cortes, como uma barra de comprimento n
- ▶ os outros $n - 1$ argumentos correspondem ao valor máximo de venda obtido através do corte da barra em dois pedaços de comprimento i e $n - i$, para cada $i = 1, 2, \dots, n - 1$, e então cortar otimamente esses pedaços, obtendo valores r_i e r_{n-i}

Subestrutura ótima

- ▶ para solucionar o problema original de comprimento n , resolvemos problemas menores do mesmo tipo, mas de comprimentos menores
- ▶ uma vez que fazemos o primeiro corte, podemos considerar os dois pedaços resultantes como instâncias independentes do problema do corte da barra de aço
- ▶ a solução ótima do problema todo incorpora soluções ótimas para os dois subproblemas relacionados, maximizando o valor de venda para cada um dos dois pedaços
- ▶ dizemos que o problema do corte da barra de aço exibe **subestrutura ótima**: soluções ótimas para um problema incorporam soluções ótimas para os subproblemas relacionados, que podemos solucionar independentemente

Simplificação

- ▶ simplificamos uma decomposição de tal forma que consista de um primeiro pedaço à esquerda de comprimento i e um pedaço remanescente de comprimento $n - i$
- ▶ apenas o pedaço remanescente (segundo) pode ser dividido
- ▶ a decomposição de uma barra de comprimento n é composta de um primeiro pedaço mais a decomposição do pedaço remanescente

Simplificação

- ▶ a solução sem cortes pode ser vista como o primeiro pedaço de comprimento $i = n$ e valor de venda p_n e o pedaço remanescente com comprimento 0 com valor de venda correspondente $r_0 = 0$
- ▶ obtemos então a seguinte versão da expressão (1):

$$r_n = \max_{1 \leq i \leq n} (p_i + r_{n-i})$$

- ▶ nessa formulação, uma solução ótima incorpora a solução de somente um subproblema relacionado (o remanescente)

Versão recursiva

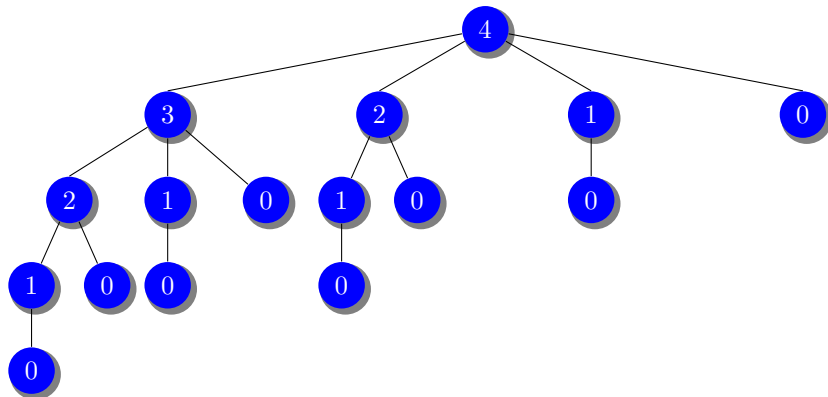
```
CUT-ROD( $p, n$ )  
01.  se  $n = 0$   
02.    devolva 0  
03.   $q \leftarrow -\infty$   
04.  para  $i \leftarrow 1$  até  $n$   
05.     $q \leftarrow \max(q, p[i] + \text{CUT-ROD}(p, n - i))$   
06.  devolva  $q$ 
```

Versão recursiva

- ▶ se codificamos o algoritmo `CUT-ROD` em uma linguagem de programação e o executamos em um computador, percebemos que o programa fica lento quando o tamanho da entrada torna-se moderadamente grande
- ▶ para $n = 40$, esse programa deve gastar algo em torno de uma hora
- ▶ a cada incremento de 1 em n , o tempo de execução do programa dobra, aproximadamente
- ▶ isso devido às repetidas chamadas recursivas de `CUT-ROD` com os mesmos parâmetros

Versão recursiva

Exemplo de chamada de CUT-ROD com $n = 4$



Versão recursiva

- ▶ seja $T(n)$ o número total de chamadas ao CUT-ROD quando seu segundo parâmetro vale n
- ▶ essa expressão é equivalente ao número de nós em uma subárvore cuja raiz é rotulada com n na árvore de recursão
- ▶ assim

$$T(n) = \begin{cases} 1, & \text{se } n = 0 \\ 1 + \sum_{j=0}^{n-1} T(j), & \text{se } n \geq 1 \end{cases}$$

- ▶ podemos mostrar que $T(n) = \Omega(2^n)$

Programação dinâmica

- ▶ tendo observado que a solução recursiva ingênua é ineficiente porque soluciona os mesmos subproblemas repetidamente, fazemos com que cada problema seja solucionado uma única vez, gravando sua solução
- ▶ se precisamos da solução desse subproblema de novo posteriormente, podemos apenas consultá-la ao invés de recomputá-la
- ▶ programação dinâmica usa memória adicional para economizar tempo de computação, que pode ser dramática: uma solução de tempo exponencial pode ser transformada em uma solução de tempo polinomial

Programação dinâmica

- ▶ existem usualmente duas formas equivalentes de implementar a técnica de programação dinâmica
- ▶ a primeira delas é chamada *top-down with memoization*: escrevemos um algoritmo recursivo, modificado para salvar os resultado de cada subproblema
- ▶ e a segunda técnica é chamada *bottom-up method*: solucionar qualquer subproblema particular depende somente da solução de subproblemas “menores”

Top-down with memoization

MEMOIZED-CUT-ROD(p, n)

01. seja $r[0..n]$ um novo vetor
02. **para** $i \leftarrow 0$ **até** n
03. $r[i] \leftarrow -\infty$
04. **devolva** MEMOIZED-CUT-ROD-AUX(p, n, r)

MEMOIZED-CUT-ROD-AUX(p, n, r)

01. **se** $r[n] \geq 0$
02. **devolva** $r[n]$
03. **se** $n = 0$
04. $q \leftarrow 0$
05. **senão**
06. $q \leftarrow -\infty$
07. **para** $i \leftarrow 1$ **até** n
08. $q \leftarrow \max(q, p[i] + \text{MEMOIZED-CUT-ROD-AUX}(p, n - i, r))$
09. $r[n] \leftarrow q$
10. **devolva** q

Bottom-up method

BOTTOM-UP-CUT-ROD(p, n)

01. seja $r[0..n]$ um novo vetor

02. $r[0] \leftarrow 0$

03. **para** $j \leftarrow 1$ **até** n

04. $q \leftarrow -\infty$

05. **para** $i \leftarrow 1$ **até** j

06. $q \leftarrow \max(q, p[i] + r[j - i])$

07. $r[j] \leftarrow q$

08. **devolva** $r[n]$

Programação dinâmica

- ▶ ambos algoritmos MEMOIZED-CUT-ROD e BOTTOM-UP-CUT-ROD têm tempo de execução $\Theta(n^2)$

Problema

Problema da multiplicação de uma sequência de matrizes

Dada uma sequência $\langle A_1, A_2, \dots, A_n \rangle$ de n matrizes, onde a matriz A_i tem dimensões $p_{i-1} \times p_i$, para $i = 1, 2, \dots, n$, encontre uma parentização de $A_1 A_2 \cdots A_n$ que minimize o número de multiplicações escalares

Propriedades do problema

- ▶ Uma multiplicação de matrizes é **completamente parentizada** se é composta por uma única matriz ou é a multiplicação de duas multiplicações de matrizes completamente parentizadas, envolvida por parênteses
- ▶ multiplicação de matrizes é associativa

Propriedades do problema

- ▶ por exemplo, se a sequência de matrizes é $\langle A_1, A_2, A_3, A_4 \rangle$, a multiplicação $A_1 A_2 A_3 A_4$ pode ser completamente parentizada de cinco maneiras diferentes:

$$(A_1(A_2(A_3A_4)))$$

$$(A_1((A_2A_3)A_4))$$

$$((A_1A_2)(A_3A_4))$$

$$((A_1(A_2A_3))A_4)$$

$$(((A_1A_2)A_3)A_4)$$

Propriedades do problema

- ▶ a forma como parentizamos uma sequência de matrizes pode ter um impacto dramático no custo da avaliação da multiplicação
- ▶ duas matrizes A e B são **compatíveis** se o número de colunas de A é igual ao número de linhas de B
- ▶ se A e B são compatíveis e A é uma matriz de dimensões $p \times q$ e B é uma matriz de dimensões $q \times r$, então o tempo para computar a matriz resultante C de dimensões $p \times r$ é pqr , usando um algoritmo usual para multiplicação de duas matrizes

Propriedades do problema

- ▶ por exemplo, considere a sequência de três matrizes $\langle A_1, A_2, A_3 \rangle$
- ▶ suponha que as dimensões dessas matrizes são 10×100 , 100×5 e 5×50 , respectivamente
- ▶ se a multiplicação é realizada de acordo com a parentização $((A_1 A_2) A_3)$ temos $10 \cdot 100 \cdot 5 = 5000$ e $10 \cdot 5 \cdot 50 = 2500$, totalizando 7500 multiplicações escalares
- ▶ se a parentização $(A_1 (A_2 A_3))$ é considerada, realizamos $100 \cdot 5 \cdot 50 = 25000$ e $10 \cdot 100 \cdot 50 = 50000$, totalizando 75000 multiplicações escalares

Propriedades do problema

- ▶ no problema da multiplicação de uma sequência de matrizes, não multiplicamos matrizes
- ▶ nosso objetivo é determinar a ordem para a multiplicação de matrizes que tem o menor custo

Propriedades do problema

- ▶ denote por $P(n)$ o número de parentizações alternativas de uma sequência de n matrizes
- ▶ quando $n = 1$, existe apenas uma matriz na sequência e portanto uma única forma de parentizar completamente a multiplicação das matrizes
- ▶ quando $n \geq 2$, a multiplicação de uma sequência de matrizes completamente parentizada é a multiplicação de duas subsequências de matrizes completamente parentizadas
- ▶ a “quebra” dessas subsequências pode ocorrer entre a k -ésima e a $(k + 1)$ -ésima matrizes, para qualquer $k = 1, 2, \dots, n - 1$
- ▶ assim, obtemos

$$P(n) = \begin{cases} 1, & \text{se } n = 1, \\ \sum_{k=1}^{n-1} P(k)P(n-k), & \text{se } n \geq 2. \end{cases}$$

- ▶ $P(n) = \Omega(2^n)$

Programação dinâmica

Passo 1: estrutura de uma parentização ótima

- ▶ denote por $A_{i..j}$, onde $i \leq j$, a matriz que resulta da avaliação da multiplicação de $A_i A_{i+1} \cdots A_j$
- ▶ se $i < j$ então qualquer parentização da multiplicação $A_i A_{i+1} \cdots A_j$ deve quebrar a sequência entre A_k e A_{k+1} para algum inteiro k no intervalo $i \leq k < j$
- ▶ ou seja, para algum valor k , devemos primeiro computar as matrizes $A_{i..k}$ e $A_{k+1..j}$ e então multiplicá-las para produzir a matriz resultante $A_{i..j}$

Programação dinâmica

Passo 1: estrutura de uma parentização ótima

- ▶ suponha que uma parentização ótima de $A_i A_{i+1} \cdots A_j$ quebra essa sequência entre A_k e A_{k+1}
- ▶ a parentização da subsequência $A_i A_{i+1} \cdots A_k$ dentro da parentização ótima de $A_i A_{i+1} \cdots A_j$ deve ser ótima
- ▶ por que?
- ▶ argumento similar é usado para a subsequência $A_{k+1} A_{k+2} \cdots A_j$

Programação dinâmica

Passo 2: uma solução recursiva

- ▶ determinar o custo mínimo de uma parentização da sequência $A_i A_{i+1} \cdots A_j$ para $1 \leq i \leq j \leq n$
- ▶ seja $m[i, j]$ o número mínimo de multiplicações escalares necessárias para computar a matriz $A_{i..j}$
- ▶ para o problema completo, o custo mínimo para computar $A_{1..n}$ é então $m[1, n]$

Programação dinâmica

Passo 2: uma solução recursiva

- ▶ se $i = j$ então o problema é trivial: a sequência consiste de apenas uma matriz $A_{i..i} = A_i$, de tal forma que nenhuma multiplicação escalar é necessária
- ▶ assim $m[i, i] = 0$ para $i = 1, 2, \dots, n$

Programação dinâmica

Passo 2: uma solução recursiva

- ▶ para computar $m[i, j]$ quando $i < j$, observamos a estrutura de uma solução ótima (Passo 1)
- ▶ suponha que a parentização ótima quebre a multiplicação $A_i A_{i+1} \cdots A_j$ entre A_k e A_{k+1} , onde $i \leq k < j$
- ▶ então, $m[i, j]$ é igual ao custo mínimo para computar os subprodutos $A_{i..k}$ e $A_{k+1..j}$ mais o custo para multiplicar as duas matrizes resultantes
- ▶ se a matriz A_i tem dimensão $p_{i-1} \times p_i$, a computação da multiplicação das matrizes $A_{i..k} \cdot A_{k+1..j}$ realiza $p_{i-1} p_k p_j$ multiplicações escalares
- ▶ assim, obtemos

$$m[i, j] = m[i, k] + m[k + 1, j] + p_{i-1} p_k p_j$$

Programação dinâmica

Passo 2: uma solução recursiva

- ▶ essa equação recursiva supõe que sabemos o valor de k , mas não sabemos de fato
- ▶ existem $j - i$ possíveis valores para k , a saber, $k = i, i + 1, \dots, j - 1$
- ▶ como a parentização ótima deve usar um desses valores para k , precisamos verificar todos para encontrar o melhor deles

Programação dinâmica

Passo 2: uma solução recursiva

- ▶ assim, a definição recursiva para o custo mínimo da parentização da multiplicação $A_i A_{i+1} \cdots A_j$ é dada por:

$$m[i, j] = \begin{cases} 0, & \text{se } i = j, \\ \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + p_{i-1} p_k p_j\}, & \text{se } i < j. \end{cases}$$

Programação dinâmica

Passo 2: uma solução recursiva

- ▶ os valores $m[i, j]$ fornecem os custos de soluções ótimas para os subproblemas
- ▶ para ajudar a manter o controle de como construir uma solução ótima, definimos $s[i, j]$ que armazena o valor de k onde podemos quebrar o produto $A_i A_{i+1} \cdots A_j$ para obter uma parentização ótima
- ▶ ou seja, $s[i, j]$ é igual a k quando
$$m[i, j] = m[i, k] + m[k + 1, j] + p_{i-1} p_k p_j$$

Programação dinâmica

Passo 3: computação dos custos ótimos

- ▶ a implementação direta da fórmula recursiva gera um algoritmo de tempo exponencial
- ▶ temos relativamente poucos subproblemas a solucionar: um problema para cada escolha de i e j satisfazendo $1 \leq i \leq j \leq n$, ou $\binom{n}{2} + n = \Theta(n^2)$ no total
- ▶ computamos o custo ótimo usando uma estratégia tabular, de baixo para cima (*bottom-up method*)

Algoritmo MATRIX-CHAIN-ORDER

```

MATRIX-CHAIN-ORDER(p)
01.   $n \leftarrow p.length - 1$ 
02.  para  $i \leftarrow 1$  até  $n$ 
03.     $m[i, i] \leftarrow 0$ 
04.  para  $l \leftarrow 2$  até  $n$ 
05.    para  $i \leftarrow 1$  até  $n - l + 1$ 
06.       $j \leftarrow i + l - 1$ 
07.       $m[i, j] \leftarrow \infty$ 
08.      para  $k \leftarrow i$  até  $j - 1$ 
09.         $q \leftarrow m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j$ 
10.        se  $q < m[i, j]$ 
11.           $m[i, j] \leftarrow q$ 
12.           $s[i, j] \leftarrow k$ 
13.  devolva  $m$  e  $s$ 

```

Exemplo de execução

Passo 3: computação dos custos ótimos

Exemplo de execução:

- ▶ $n = 6$ e as seguintes dimensões das matrizes:

matriz	dimensão
A_1	30×35
A_2	35×15
A_3	15×5
A_4	5×10
A_5	10×20
A_6	20×25

Exemplo de execução

m	1	2	3	4	5	6
1	0	15.750	7.875	9.375	11.875	15.125
2		0	2.625	4.375	7.125	10.500
3			0	750	2.500	5.375
4				0	1.000	3.500
5					0	5.000
6						0

s	1	2	3	4	5	6
1		1	1	3	3	3
2			2	3	3	3
3				3	3	3
4					4	5
5						5
6						

Tempo de execução

Passo 3: computação dos custos ótimos

por simples inspeção no código do algoritmo `MATRIX-CHAIN-ORDER`, concluímos que seu tempo de execução é $O(n^3)$

Construção de uma solução ótima

Passo 4: construção de uma solução ótima

- ▶ cada entrada $s[i, j]$ armazena o valor de k tal que a parentização ótima de $A_i A_{i+1} \cdots A_j$ quebra a sequência entre A_k e A_{k+1}
- ▶ sabemos que a matriz final resultante da computação ótima de $A_{1..n}$ é $A_{1..s[1,n]} A_{s[1,n]+1..n}$

Algoritmo PRINT-OPTIMAL-PARENS

```
PRINT-OPTIMAL-PARENS( $s, i, j$ )  
01.  se  $i = j$   
02.    imprima " $A_i$ "  
03.  senão  
04.    imprima "("  
05.    PRINT-OPTIMAL-PARENS( $s, i, s[i, j]$ )  
06.    PRINT-OPTIMAL-PARENS( $s, s[i, j] + 1, j$ )  
07.    imprima ")"
```

Motivação

- ▶ em aplicações em Biologia Molecular, frequentemente queremos comparar dois ou mais DNAs de organismos diferentes
- ▶ uma fita de DNA consiste de uma cadeia de moléculas chamadas **bases** (adenina, guanina, citosina e timina)
- ▶ representando cada uma dessas bases com a primeira letra de seus nomes, uma fita de DNA pode ser expressa como uma cadeia de caracteres sobre o alfabeto $\{A, C, G, T\}$
- ▶ por exemplo, o DNA de um organismo pode ser $S_1 = \text{ACCGGTCGAGTGCGCGGAAGCCGGCCGAA}$ e de um outro $S_2 = \text{GTCGTTCGGAATGCCGTTGCTCTGTAAA}$

Motivação

- ▶ a comparação de duas fitas de DNA tem como um dos objetivos determinar quão “similares” elas são
- ▶ exemplos de similaridade: se uma fita é uma subcadeia da outra, se o número de trocas de bases necessárias para transformar uma sequência na outra é pequeno, encontrar uma terceira fita S_3 tal que as bases que ocorrem em S_1 e S_2 (bases devem ocorrer na mesma ordem, mas não necessariamente de forma consecutiva), etc

Definições iniciais

- ▶ uma subsequência de uma dada sequência é a própria sequência excluindo zero ou mais elementos
- ▶ dada uma sequência $X = \langle x_1, x_2, \dots, x_m \rangle$, uma outra sequência $Z = \langle z_1, z_2, \dots, z_k \rangle$ é uma **subsequência** de X se existe uma sequência crescente $\langle i_1, i_2, \dots, i_k \rangle$ de índices de X tal que para todo $j = 1, 2, \dots, k$, temos $x_{i_j} = z_j$
- ▶ por exemplo, $Z = \langle C, G, T, C \rangle$ é uma subsequência de $X = \langle A, C, G, C, T, A, C \rangle$ com sequência correspondente de índices $\langle 2, 3, 5, 7 \rangle$

Definições iniciais

- ▶ dadas duas sequências X e Y , dizemos que uma sequência Z é uma **subsequência comum** de X e Y se Z é subsequência de X e Y
- ▶ por exemplo, se $X = \langle A, C, G, C, T, A, C \rangle$ e $Y = \langle C, T, G, A, C, A \rangle$, a sequência $\langle C, G, A \rangle$ é uma subsequência comum de X e Y , assim como $\langle C, G, C, A \rangle$
- ▶ no exemplo anterior, $\langle C, G, C, A \rangle$ é uma **maior subsequência comum** (LCS) de X e Y

Problema

Problema da maior subsequência comum (LCS)

Dadas duas sequências $X = \langle x_1, x_2, \dots, x_m \rangle$ e $Y = \langle y_1, y_2, \dots, y_n \rangle$, encontrar uma subsequência comum de X e Y de maior comprimento

Algoritmo ingênuo

- ▶ a técnica de força bruta para solucionar o problema LCS é enumerar todas as subsequências de X e verificar se cada subsequência é também uma subsequência de Y , mantendo registro da maior subsequência encontrada
- ▶ cada subsequência de X corresponde a um subconjunto de índices de $\{1, 2, \dots, m\}$ de X
- ▶ existem 2^m subsequências de X e, assim, essa estratégia requer tempo exponencial

Programação dinâmica

Passo 1: caracterização de uma maior subsequência comum

- ▶ o problema LCS tem a propriedade da subestrutura ótima
- ▶ as classes naturais de subproblemas correspondem a pares de “prefixos” das duas sequências de entrada
- ▶ dada uma sequência $X = \langle x_1, x_2, \dots, x_m \rangle$, o i -ésimo **prefixo** de X , para $i = 0, 1, \dots, m$, é definido como $X_i = \langle x_1, x_2, \dots, x_i \rangle$, com X_0 como a sequência vazia

Programação dinâmica

Passo 1: caracterização de uma maior subsequência comum

Teorema (Subestrutura ótima de uma LCS)

Sejam $X = \langle x_1, x_2, \dots, x_m \rangle$ e $Y = \langle y_1, y_2, \dots, y_n \rangle$ sequências e seja $Z = \langle z_1, z_2, \dots, z_k \rangle$ uma LCS de X e Y .

1. Se $x_m = y_n$, então $z_k = x_m = y_n$ e Z_{k-1} é uma LCS de X_{m-1} e Y_{n-1}
2. Se $x_m \neq y_n$, então $z_k \neq x_m$ implica que Z é uma LCS de X_{m-1} e Y
3. Se $x_m \neq y_n$, então $z_k \neq y_n$ implica que Z é uma LCS de X e Y_{n-1}

Programação dinâmica

Passo 2: uma solução recursiva

- ▶ o Teorema implica que existe um ou dois subproblemas a examinar durante a busca por uma LCS de $X = \langle x_1, x_2, \dots, x_m \rangle$ e $Y = \langle y_1, y_2, \dots, y_n \rangle$
- ▶ se $x_m = y_n$, devemos encontrar uma LCS de X_{m-1} e Y_{n-1} ; adicionando $x_m = y_n$ a esse LCS, resulta em uma LCS de X e Y
- ▶ se $x_m \neq y_n$ então devemos resolver dois subproblemas: encontrar uma LCS de X_{m-1} e Y e encontrar uma LCS de X e Y_{n-1} ; a maior delas é uma LCS de X e Y

Programação dinâmica

Passo 2: uma solução recursiva

- ▶ defina $c[i, j]$ como o comprimento de uma LCS das sequências X_i e Y_j
- ▶ se $i = 0$ ou $j = 0$, uma das sequências tem comprimento 0 e assim a LCS tem comprimento 0
- ▶ a subestrutura ótima do problema LCS fornece a fórmula recursiva:

$$c[i, j] = \begin{cases} 0, & \text{se } i = 0 \text{ ou } j = 0, \\ c[i-1, j-1] + 1, & \text{se } i, j > 0 \text{ e } x_i = y_j, \\ \max(c[i, j-1], c[i-1, j]), & \text{se } i, j > 0 \text{ e } x_i \neq y_j. \end{cases}$$

Programação dinâmica

Passo 3: computação do comprimento de uma LCS

- ▶ baseado na fórmula recursiva é fácil escrever um algoritmo de tempo exponencial para computar o comprimento de uma LCS de duas sequências dadas
- ▶ no entanto, como existem apenas $\Theta(mn)$ subproblemas, podemos usar programação dinâmica para computar as soluções de baixo para cima (*bottom up method*)
- ▶ mantemos também a tabela $b[1..m, 1..n]$ para simplificar a construção de uma solução ótima: a posição $b[i, j]$ aponta para a entrada na tabela correspondente solução do subproblema ótimo escolhida quando da computação de $c[i, j]$

Algoritmo LCS-LENGTH

```

LCS-LENGTH( $X, Y$ )
01.   $m \leftarrow X.length$ 
02.   $n \leftarrow Y.length$ 
03.  para  $i \leftarrow 1$  até  $m$ 
04.     $c[i, 0] \leftarrow 0$ 
05.  para  $j \leftarrow 0$  até  $n$ 
06.     $c[0, j] \leftarrow 0$ 
07.  para  $i \leftarrow 1$  até  $m$ 
08.    para  $j \leftarrow 1$  até  $n$ 
09.      se  $x_i = y_j$ 
10.         $c[i, j] \leftarrow c[i - 1, j - 1] + 1$ 
11.         $b[i, j] \leftarrow ' \nwarrow '$ 
12.      senão se  $c[i - 1, j] \geq c[i, j - 1]$ 
13.         $c[i, j] \leftarrow c[i - 1, j]$ 
14.         $b[i, j] \leftarrow ' \uparrow '$ 
15.      senão
16.         $c[i, j] \leftarrow c[i, j - 1]$ 
17.         $b[i, j] \leftarrow ' \leftarrow '$ 
18.  devolva  $c$  e  $b$ 
    
```

Exemplo de execução

j		0	1	2	3	4	5	6
i		y_j	B	D	C	A	B	A
0	x_i	0	0	0	0	0	0	0
1	A	0	↑ 0	↑ 0	↑ 0	↖ 1	← 1	← 1
2	B	0	↘ 1	← 1	← 1	↑ 1	↖ 2	← 2
3	C	0	↑ 1	↑ 1	↘ 2	← 2	↑ 2	↑ 2
4	B	0	↖ 1	↑ 1	↑ 2	↑ 2	↘ 3	← 3
5	D	0	↑ 1	↖ 2	↑ 2	↑ 2	↑ 3	↑ 3
6	A	0	↑ 1	↑ 2	↑ 2	↖ 3	↑ 3	↘ 4
7	B	0	↖ 1	↑ 2	↑ 2	↑ 3	↖ 4	↑ 4

Tempo de execução

Passo 3: computação do comprimento de uma LCS

por simples inspeção no código do algoritmo `LCS-LENGTH`, concluímos que seu tempo de execução é $O(mn)$

Construção de uma solução ótima

Passo 4: construção de uma LCS

- ▶ a tabela b devolvida pelo algoritmo `LCS-LENGTH` é usada para construir uma LCS de $X = \langle x_1, x_2, \dots, x_m \rangle$ e $Y = \langle y_1, y_2, \dots, y_n \rangle$
- ▶ sempre que encontramos uma $\nwarrow$ na entrada $b[i, j]$, isso implica que $x_i = y_j$ é um elemento da LCS
- ▶ os elementos da LCS são encontrados em ordem reversa usando essa estratégia

Algoritmo PRINT-LCS

```

PRINT-LCS( $b, X, i, j$ )
01.   se  $i = 0$  ou  $j = 0$ 
02.     pare
03.   se  $b[i, j] \leftarrow ' \nwarrow '$ 
04.     PRINT-LCS( $b, X, i - 1, j - 1$ )
05.     imprima  $x_i$ 
06.   senão se  $b[i, j] = ' \uparrow '$ 
07.     PRINT-LCS( $b, X, i - 1, j$ )
08.   senão
09.     PRINT-LCS( $b, X, i, j - 1$ )
    
```

Ingredientes básicos

Quando devemos desenvolver uma solução usando programação dinâmica?

- ▶ subestrutura ótima
- ▶ sobreposição de subproblemas

Subestrutura ótima

- ▶ um problema apresenta **subestrutura ótima** se uma solução ótima para o problema contém internamente soluções ótimas para subproblemas
- ▶ por exemplo, o valor máximo de venda r_n de uma barra de aço de comprimento n , no problema do corte de uma barra de aço, contém internamente o valor máximo de venda r_{n-i} de uma barra de aço de comprimento $n - i$, para algum i

Subestrutura ótima

Devemos seguir os seguintes passos para descobrir a subestrutura ótima de um problema:

1. mostrar que uma solução para o problema consiste de fazer uma escolha; fazer essa escolha implica em um ou mais subproblemas a serem solucionados
2. supor que, para um problema, é dada a escolha que leva a uma solução ótima; não é necessário, ainda, saber como determinar essa escolha
3. dada essa escolha, determinar quais subproblemas ocorrem e como caracterizar da melhor forma o espaço de subproblemas resultante
4. mostrar que as soluções para os subproblemas usadas na solução ótima para o problema devem, elas mesmas, ser ótimas; podemos supor que uma das soluções para um subproblema não é ótima e então obtemos uma contradição

Subestrutura ótima

Subestrutura ótima varia entre os domínios dos problemas de duas formas:

- ▶ quantos problemas são usados em uma solução ótima do problema original, e
- ▶ quantas escolhas temos para determinar qual(is) subproblema(s) usar em uma solução ótima

Subestrutura ótima

- ▶ no problema do corte de uma barra de aço, uma solução ótima usa apenas um subproblema, mas devemos considerar n escolhas para determinar uma solução ótima
- ▶ para encontrar o valor máximo de venda r_n de uma barra de aço de comprimento n , usamos o valor máximo de venda de uma barra de comprimento r_{n-1} **ou** o valor máximo de venda de uma barra de comprimento r_{n-2} **ou** ... **ou** o valor máximo de venda de uma barra de comprimento r_0
- ▶ seja qual for o pedaço de barra usado, ele representa o subproblema que devemos resolver otimamente

Subestrutura ótima

- ▶ programação dinâmica usa a subestrutura ótima:
 - ▶ de cima para baixo (*top-down*) com *memoization*, ou
 - ▶ de baixo para cima (*bottom-up*)

Sobreposição de subproblemas

- ▶ o espaço de subproblemas deve ser “pequeno”, no sentido que um algoritmo recursivo para o problema soluciona os mesmos subproblemas diversas vezes, ao invés de sempre gerar novos subproblemas
- ▶ quando um algoritmo recursivo revisita o mesmo (sub)problema várias vezes, dizemos que o problema de otimização tem **sobreposição de subproblemas**
- ▶ algoritmos de programação dinâmica tipicamente aproveitam-se da sobreposição de subproblemas solucionando cada subproblema uma única vez e então armazenando o valor da solução em uma tabela onde pode ser recuperado em tempo constante quando necessário

Sobreposição de subproblemas

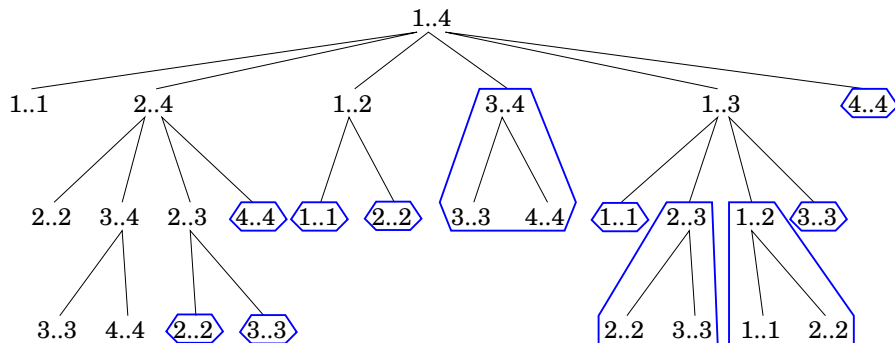
Relembrando o exemplo da multiplicação de uma sequência de matrizes

```

RECURSIVE-MATRIX-CHAIN( $p, i, j$ )
01.  se  $i = j$ 
02.    devolva 0
03.   $m[i, j] \leftarrow \infty$ 
04.  para  $k \leftarrow i$  até  $j - 1$ 
05.     $q \leftarrow$  RECURSIVE-MATRIX-CHAIN( $p, i, k$ )
        + RECURSIVE-MATRIX-CHAIN( $p, k + 1, j$ )
        +  $p_{i-1}p_kp_j$ 
06.    se  $q < m[i, j]$ 
07.       $m[i, j] \leftarrow q$ 
08.  devolva  $m[i, j]$ 
  
```

Sobreposição de subproblemas

Árvore de recursão para a chamada `RECURSIVE-MATRIX-CHAIN( $p, 1, 4$ )`



Sobreposição de subproblemas

- ▶ tempo de execução do algoritmo `RECURSIVE-MATRIX-CHAIN` é $\Omega(2^n)$
- ▶ tempo de execução do algoritmo `MATRIX-CHAIN-ORDER`, usando programação dinâmica, é $\Theta(n^3)$

Memoization

- ▶ como vimos, especialmente no problema do corte da barra de metal, existe uma técnica alternativa de programação dinâmica que tem a eficiência do método *bottom-up*, mesmo mantendo a estratégia *top-down*
- ▶ como no método *bottom-up*, mantemos uma tabela com as soluções para os subproblemas, mas a estrutura de controle para preencher a tabela é mais parecida com um algoritmo recursivo

Memoization

```

MEMOIZED-MATRIX-CHAIN( $p$ )
01.   $n \leftarrow p.length - 1$ 
02.  seja  $m[1..n, 1..n]$  uma nova matriz
03.  para  $i \leftarrow 1$  até  $n$ 
04.    para  $j \leftarrow 1$  até  $n$ 
05.       $m[i, j] \leftarrow \infty$ 
06.  devolva LOOKUP-CHAIN( $m, p, 1, n$ )
  
```

Memoization

```

LOOKUP-CHAIN( $m, p, i, j$ )
01.  se  $m[i, j] < \infty$ 
02.    devolva  $m[i, j]$ 
03.  se  $i = j$ 
04.     $m[i, j] \leftarrow 0$ 
05.  senão
06.    para  $k \leftarrow i$  até  $j - 1$ 
07.       $q = \text{LOOKUP-CHAIN}(m, p, i, k)$ 
         $+ \text{LOOKUP-CHAIN}(m, p, k + 1, j)) + p_{i-1}p_kp_j$ 
08.      se  $q < m[i, j]$ 
09.         $m[i, j] \leftarrow q$ 
10.  devolva  $m[i, j]$ 

```

Memoization

- ▶ como o algoritmo de programação dinâmica MATRIX-CHAIN-ORDER que usa o método *bottom-up*, o algoritmo MEMOIZED-MATRIX-CHAIN tem tempo $O(n^3)$
- ▶ a linha 5 do algoritmo MEMOIZED-MATRIX-CHAIN é executada $O(n^2)$ vezes
- ▶ as chamadas ao algoritmo LOOKUP-CHAIN são organizadas em dois tipos:
 1. chamadas em que $m[i, j] = \infty$, tais que as linhas 3–9 são executadas, e
 2. chamadas em que $m[i, j] < \infty$, tais que o algoritmo LOOKUP-CHAIN simplesmente termina na linha 2

Memoization

- ▶ existem $\Theta(n^2)$ chamadas do tipo 1
- ▶ todas as chamadas do tipo 2 são feitas como chamadas recursivas de chamadas do tipo 1
- ▶ sempre que uma dada chamada de LOOKUP-CHAIN faz chamadas recursivas, faz $O(n)$ delas
- ▶ portanto, existem $O(n^3)$ chamadas do tipo 2 no total
- ▶ cada chamada do tipo 2 tem tempo $O(1)$ e cada chamada do tipo 1 tem tempo $O(n)$ mais o tempo gasto nas chamadas recursivas
- ▶ o tempo total é, portanto, $O(n^3)$

Memoization

- ▶ assim, podemos solucionar o problema da multiplicação de uma sequência de matrizes usando um algoritmo de programação dinâmica com estratégia *top-down with memoization* ou um algoritmo de programação dinâmica com o *bottom-up method*: ambos têm tempo $O(n^3)$
- ▶ ambos algoritmos se aproveitam da propriedade da sobreposição de subproblemas
- ▶ existem apenas $\Theta(n^2)$ subproblemas distintos no total e ambos esses métodos computam a solução para cada subproblema somente uma vez
- ▶ na prática, se todos os subproblemas precisam ser solucionados ao menos uma vez, um algoritmo de programação dinâmica usando o *bottom-up method* tende a superar por uma constante seu correspondente usando a estratégia *top-down with memoization*

Exercícios

1. Um segmento de um vetor $A[p..r]$ é qualquer subvetor da forma $A[i..k]$, com $p \leq i \leq k \leq r$. A condição $i \leq k$ garante que o segmento não é vazio. A **soma** de um segmento $A[i..k]$ é o número $A[i] + A[i+1] + \dots + A[k]$. A **solidez** de um vetor $A[p..r]$ é a soma de um segmento de soma máxima.

Problema do Segmento de Soma Máxima: Calcular a solidez de um vetor $A[p..r]$ de números inteiros.

Descreva um algoritmo de programação dinâmica que soluciona o problema do segmento de soma máxima.

Exercícios

15.1-1 Mostre que a igualdade $T(n) = 2^n$ decorre da recorrência

$$T(n) = \begin{cases} 1, & \text{se } n = 0 \\ 1 + \sum_{j=0}^{n-1} T(j), & \text{se } n \geq 1 \end{cases}$$

15.1-2 Mostre por meio de um contraexemplo que a seguinte estratégia “gulosa” nem sempre determina uma forma ótima de cortar barras. Defina a **densidade** de uma barra de comprimento i como p_i/i , isto é, seu valor por metro. A estratégia gulosa para uma barra de comprimento n corta um primeiro pedaço de comprimento i , onde $1 \leq i \leq n$, com densidade máxima. Continua-se então aplicando a estratégia gulosa ao pedaço remanescente de comprimento $n - i$.

Exercícios

15.2-1 Encontre uma parentização ótima da multiplicação de uma sequência de matrizes cuja sequência de dimensões é $\langle 5, 10, 3, 12, 5, 50, 6 \rangle$.

15.2-3 Use o método da substituição para mostrar que a solução da recorrência

$$P(n) = \begin{cases} 1, & \text{se } n = 1, \\ \sum_{k=1}^{n-1} P(k)P(n-k), & \text{se } n \geq 2, \end{cases}$$

é $\Omega(2^n)$.

15.2-4 Seja $R(i, j)$ o número de vezes que a célula $m[i, j]$ é acessada pelo algoritmo MATRIX-CHAIN-ORDER na computação dos valores de outras células da tabela. Mostre que o número total de acessos à tabela toda é

$$\sum_{i=1}^n \sum_{j=i}^n R(i, j) = \frac{n^3 - n}{3}.$$

Sugestão: a identidade $\sum_{i=1}^n i^2 = n(n+1)(2n+1)/6$ pode ser útil.

Exercícios

- 15.2-5 Mostre que uma parentização completa de uma sequência de n elementos tem exatamente $n - 1$ pares de parênteses.
- 15.3-1 Qual a forma mais eficiente de determinar o número ótimo de multiplicações escalares no problema da multiplicação de uma sequência de matrizes: enumerar todas as formas de parentização da multiplicação das matrizes e computar o número de produtos escalares para cada um deles, ou executar o algoritmo `RECURSIVE-MATRIX-CHAIN`, que implementa diretamente a recorrência? Justifique sua resposta.
- 15.3-3 Considere uma variante do problema da multiplicação de uma sequência de matrizes em que o objetivo é parentizar a sequência de matrizes de modo a maximizar, ao invés de minimizar, o número de produtos escalares. Esse problema tem subestrutura ótima?

Exercícios

- 15.3-5 Como vimos, na programação dinâmica resolvemos primeiro os subproblemas e então escolhemos quais deles usar em uma solução ótima para o problema. O professor Izkvardzen Grushteniak afirma que nem sempre precisamos resolver todos os subproblemas para encontrar uma solução ótima. Ele sugere que podemos encontrar uma solução ótima para o problema da multiplicação de uma sequência de matrizes escolhendo sempre a matriz A_k que quebra o subproduto $A_i A_{i+1} \cdots A_j$ (selecionando k que minimiza a quantidade $p_{i-1} p_k p_j$) *antes* de solucionar os subproblemas. Encontre uma entrada do problema da sequência de multiplicação de matrizes para o qual esta estratégia gulosa não fornece uma solução ótima.
- 15.4-1 Determine uma LCS de $\langle 1, 0, 0, 1, 0, 1, 0, 1 \rangle$ e $\langle 0, 1, 0, 1, 1, 0, 1, 1, 0 \rangle$.
- 15.4-5 Escreva um algoritmo de tempo $O(n^2)$ que encontra a maior subsequência crescente de uma sequência de n números.

Problemas

15-2 Maior subsequência palíndromo

Um **palíndromo** é uma cadeia de símbolos não vazia sobre algum alfabeto que é soletrada da mesma forma de trás para frente e de frente para trás. Exemplos de palíndromos são todas as cadeias de comprimento 1 e outras maiores como asa, esse, mirim, reviver.

Escreva um algoritmo eficiente para encontrar o maior palíndromo que é uma subsequência de uma dada cadeia de caracteres de entrada. Por exemplo, dada a cadeia estressante, seu algoritmo deve devolver etsste. Qual o tempo de execução do seu algoritmo?